

IF2224 FORMAL LANGUAGE AND AUTOMATA THEORY

Arion Compiler - Milestone 4: Intermediate Code and Interpreter

June 5, 2026



Made Branenda Jordhy - 13524026

Muhammad Nur Majiid - 13524028

Jason Edward Salim - 13524034

Athilla Zaidan Zidna Fann - 13524068

**School of Electrical Engineering and Informatics
Institut Teknologi Bandung
Semester II 2025/2026**

Contents

1	Theoretical Background	3
1.1	Intermediate Code Generation	3
1.2	Three Address Code dan Stack Machine	3
1.3	Interpreter	4
1.4	Runtime Safety	5
2	Design and Implementation	5
2.1	Overview Pipeline	5
2.2	Instruction Representation	6
2.3	OPR Code	6
2.4	Code Generator	7
2.5	Interpreter	8
2.6	Error Handling	9
2.7	Status Integrasi	10
3	Testing	11
3.1	Lokasi File Pengujian	11
3.2	Ringkasan Kasus Uji	11
3.3	Catatan Testcase	12
3.4	Hasil Pengujian Intermediate Code	12
3.5	Hasil Eksekusi Program	20
4	Conclusion	21
4.1	Saran	21
	Appendix	23

1 Theoretical Background

1.1 Intermediate Code Generation

Intermediate code generation adalah tahap translasi dari *Decorated AST* menuju representasi instruksi yang lebih dekat dengan mesin. Setelah Milestone 3, AST Arion sudah membawa informasi semantik seperti tipe data, indeks symbol table, lexical level, dan alamat variabel. Informasi tersebut menjadi dasar untuk mengubah ekspresi dan statement tingkat tinggi menjadi instruksi linier [1].

Gambar 1 memperlihatkan contoh *Decorated AST* untuk statement $b := a + 10$. Setiap node tidak lagi sekadar merepresentasikan struktur sintaksis, tetapi sudah dianotasi dengan tipe data hasil evaluasi dan referensi ke entri symbol table (`tab_index`). Anotasi inilah yang dibaca oleh code generator untuk memutuskan instruksi yang tepat, misalnya memilih OPR ADD untuk penjumlahan integer.

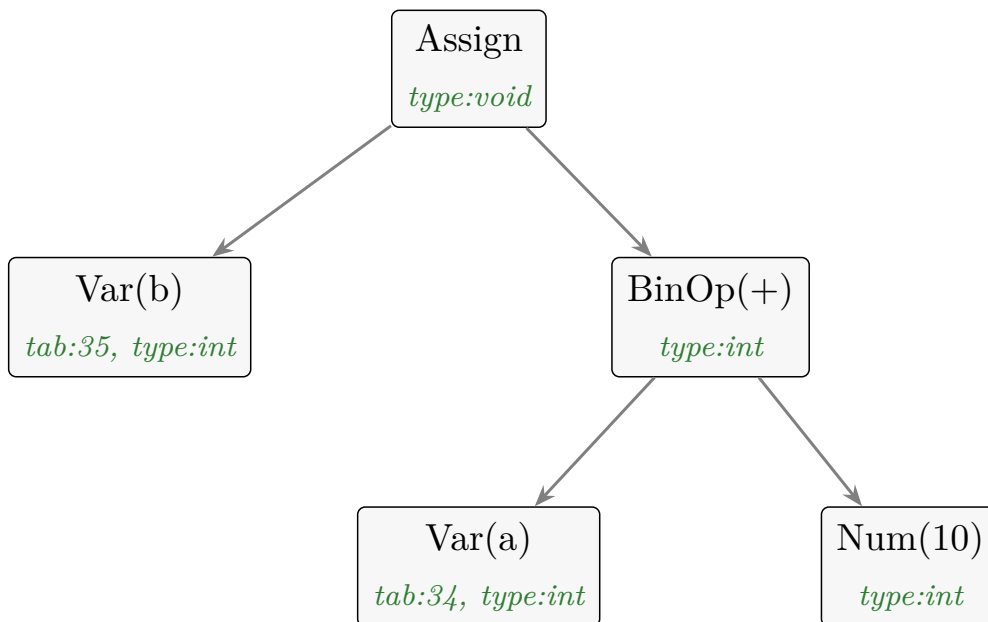


Figure 1: Decorated AST untuk $b := a + 10$; setiap node membawa anotasi tipe dan indeks symbol table sebagai masukan code generator.

Pada Milestone 4 ini, bentuk instruksi yang digunakan mengikuti model *stack machine*. Setiap ekspresi dievaluasi dengan mendorong operand ke stack, menjalankan operasi lewat instruksi OPR, lalu memakai nilai hasilnya untuk assignment, percabangan, atau output. Pendekatan ini membuat eksekusi tidak perlu lagi menelusuri pohon AST secara langsung.

1.2 Three Address Code dan Stack Machine

Secara umum, *Three Address Code* memecah ekspresi kompleks menjadi instruksi sederhana dengan jumlah operand terbatas. Pada implementasi Arion, instruksi tidak menyimpan tiga alamat eksplisit seperti $t1 := a + b$, tetapi memakai bentuk P-Code sederhana:

Line	Instruksi	Level	Operand
0	INT	0	5
1	LIT	0	5
2	STO	0	3

Maknanya tetap sama: instruksi bekerja pada alamat memori dan stack. Instruksi **LIT** mendorong literal ke stack, **LOD** membaca variabel, **STO** menulis variabel, **CAL** membentuk frame pemanggilan, **RET** membersihkan frame, **JMP/JPC** mengatur alur kontrol, dan **OPR** menjalankan operasi aritmatika, relasional, atau output.

Tabel 1 menelusuri bagaimana ekspresi $y := (3 + 4) * x$ diratakan menjadi urutan instruksi sekaligus perubahan isi stack pada setiap langkah [2]. Operand terdalam dievaluasi lebih dahulu, lalu hasil sementara ditumpuk hingga operasi terakhir menyisakan satu nilai yang disimpan oleh **STO**.

Langkah	Instruksi	Isi stack (atas → kanan)
1	LIT 3	[3]
2	LIT 4	[3, 4]
3	OPR ADD	[7]
4	LOD x	[7, val_x]
5	OPR MUL	[7 × val_x]
6	STO y	[] (hasil disimpan ke y)

Table 1: Translasi ekspresi $y := (3 + 4) * x$ menjadi urutan intermediate code beserta evolusi isi stack.

Gambar 2 menggambarkan evolusi stack tersebut secara visual untuk ekspresi $a + 10$ (dengan a bernilai 5): operand didorong, dioperasikan oleh **OPR ADD**, lalu hasilnya disimpan.

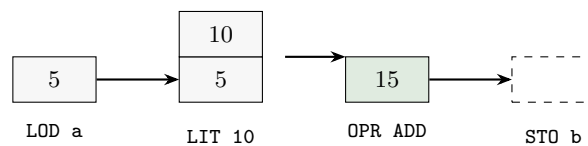


Figure 2: Aliran stack saat mengeksekusi $b := a + 10$: push operand, **OPR ADD** menggantinya dengan hasil, lalu **STO** mengosongkan stack.

1.3 Interpreter

Interpreter bertindak sebagai mesin virtual yang menjalankan daftar instruksi secara berurutan. Mesin menyimpan tiga komponen utama:

- **Instruction pointer** (ip) untuk menunjuk instruksi yang sedang dieksekusi.
- **Base pointer** (bp) untuk menunjuk awal stack frame aktif.
- **Stack** untuk menyimpan static link, dynamic link, return address, variabel lokal, parameter, operand sementara, dan hasil ekspresi.

Siklus eksekusinya mengikuti pola *fetch-decode-execute* (Gambar 3). Interpreter mengambil instruksi pada `ip`, memeriksa opcode-nya, menjalankan handler yang sesuai, lalu melanjutkan ke instruksi berikutnya kecuali terjadi lompatan atau return.

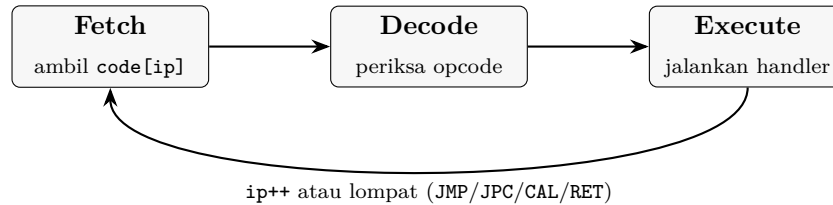


Figure 3: Siklus *fetch-decode-execute* pada interpreter Arion.

Secara konseptual, organisasi memori program mengikuti tata letak klasik pada Gambar 4: segmen *text* menyimpan kode, segmen data menyimpan variabel statis, sedangkan *stack* (tempat frame fungsi Arion hidup) tumbuh dari alamat tinggi ke rendah dan *heap* tumbuh berlawanan arah. Interpreter Arion berfokus pada segmen *stack*.

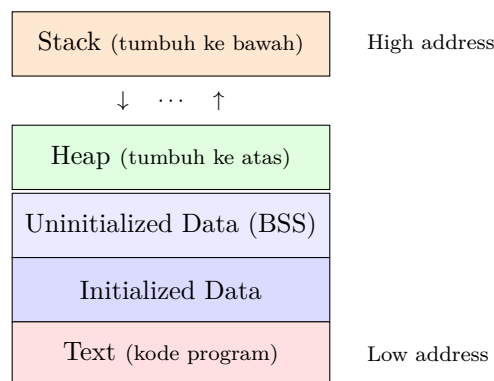


Figure 4: Tata letak memori program; interpreter Arion mengelola segmen *stack* untuk frame, operand, dan hasil sementara.

1.4 Runtime Safety

Walaupun Arion sudah melakukan pengecekan semantik secara statis, interpreter tetap perlu menjaga keamanan eksekusi. Implementasi saat ini menambahkan pengecekan *stack overflow*, *stack underflow*, akses *stack out-of-bounds*, target jump invalid, pembagian dengan nol, modulo dengan nol, serta opcode/OPR yang tidak dikenal. Dengan demikian, kesalahan runtime tidak langsung menyebabkan program C++ crash, tetapi dilaporkan melalui `RuntimeError`.

2 Design and Implementation

2.1 Overview Pipeline

Pipeline kompilasi yang menjadi dasar Milestone 4 adalah kelanjutan dari tiga milestone sebelumnya. Lexer menghasilkan token stream, parser membangun parse tree, AST builder mereduksi parse tree menjadi AST, semantic analyzer mendekorasi AST dan mengisi symbol table, lalu code generator membuat intermediate code yang dapat dijalankan interpreter.

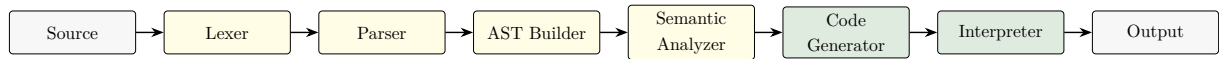


Figure 5: Pipeline lengkap Arion dari source code hingga output eksekusi.

2.2 Instruction Representation

Instruksi intermediate code didefinisikan pada `src/header/instruction.hpp` dengan empat field utama: nomor baris, opcode, level, dan operand.

```

1 struct Instruction {
2     int line;
3     OpCode op;
4     int level;
5     int operand;
6 };
  
```

Opcode yang tersedia dirangkum pada Tabel 2. Instruksi ini sesuai dengan konvensi pada spesifikasi Milestone 4, dengan tambahan struktur data C++ agar mudah dicetak dan dieksekusi.

Opcode	Fungsi
INT	Mengalokasikan ruang variabel lokal pada stack frame aktif. Frame header dibuat saat inisialisasi global atau saat CAL.
LIT	Mendorong nilai literal integer/boolean/char ke stack.
LOD	Membaca nilai variabel dari stack frame tertentu.
STO	Mengambil nilai dari puncak stack dan menyimpannya ke alamat variabel.
CAL	Melakukan pemanggilan prosedur/fungsi.
JMP	Melompat tanpa syarat ke baris instruksi tertentu.
JPC	Melompat jika kondisi pada puncak stack bernilai false/0.
OPR	Menjalankan operasi aritmatika, relasional, atau output.
RET	Keluar dari stack frame aktif dan kembali ke caller.

Table 2: Opcode intermediate code yang tersedia.

2.3 OPR Code

Instruksi OPR memakai operand sebagai kode operasi. Operasi yang diimplementasikan adalah NEG, ADD, SUB, MUL, DIV, MOD, operator relasional EQL/NEQ/LSS/GEQ/GTR/LEQ, serta WRT dan WRTLN. Nilai boolean direpresentasikan sebagai integer 0 dan 1, sehingga operasi **and** dipetakan ke perkalian dan **or** dipetakan ke penjumlahan pada code generator. Tabel 3 merangkum seluruh kode operasi OPR.

No	Op	Fungsi	No	Op	Fungsi
1	NEG	negasi unary	8	NEQ	tidak sama dengan
2	ADD	penjumlahan	9	LSS	lebih kecil
3	SUB	pengurangan	10	GEQ	lebih besar sama dengan
4	MUL	perkalian	11	GTR	lebih besar
5	DIV	pembagian integer	12	LEQ	lebih kecil sama dengan
6	MOD	modulo	13	WRT	cetak nilai
7	EQL	sama dengan	14	WRTLN	cetak nilai + newline

Table 3: Kode operasi pada instruksi OPR (operand = nomor operasi).

2.4 Code Generator

Komponen `CodeGenerator` berada pada `src/header/codegen.hpp` dan `src/lib/codegen.cpp`. Generator menerima `ASTNode*` dan `SymbolTable` hasil semantic analyzer. Sebelum menghasilkan instruksi, generator memanggil `resolveAddresses()` untuk mengisi alamat variabel pada `tab[idx].adr` berdasarkan urutan deklarasi pada setiap `btab`.

Strategi translasi dilakukan secara rekursif sesuai jenis node AST:

- `genProgram()` menghasilkan instruksi utama dan, bila ada subprogram, membuat `JMP` awal untuk melewati kode prosedur atau fungsi.
- `genAssign()` menghasilkan instruksi ekspresi kanan lalu `STO` ke alamat variabel target.
- `genIf()` memakai `JPC` yang di-*patch* setelah cabang `then/else` selesai digenerate.
- `genWhile()`, `genFor()`, dan `genRepeat()` membangun control flow dengan kombinasi label implisit berupa nomor baris, `JMP`, dan `JPC`.
- `genCall()` menangani `write/writeln` sebagai OPR `WRT/WRTLN`, dan subprogram pengguna sebagai `CAL`.

Tantangan utama pada *control flow* adalah meratakan blok bersarang menjadi instruksi linier menggunakan *label* (nomor baris) dan *jump*. Gambar 6 memperlihatkan pola translasi `if-else` dan `while`. Pada `if-else`, `JPC` melompat ke cabang `else` bila kondisi bernilai *false*, dan `JMP` melewati cabang `else` setelah `then` selesai. Pada `while`, kondisi dievaluasi ulang setiap iterasi dan `JMP` kembali ke label awal.

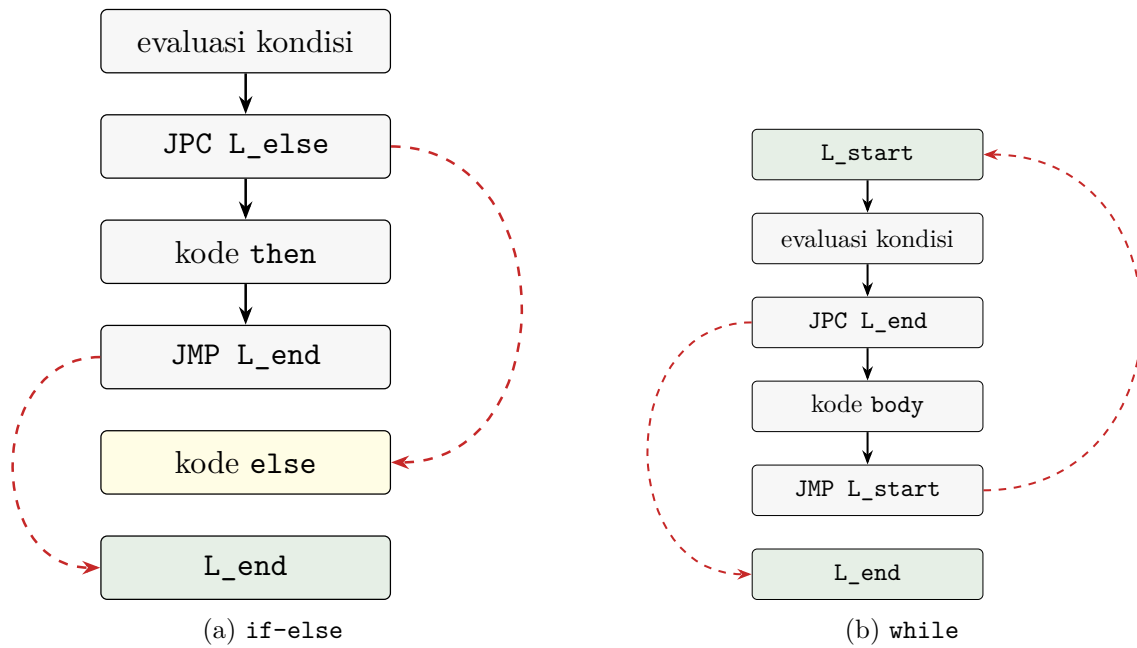


Figure 6: Pola translasi *control flow*; panah putus-putus merah adalah lompatan (JPC/JMP).

Generator juga sudah memisahkan alamat parameter dan variabel lokal. Parameter disimpan pada alamat negatif relatif terhadap base pointer, sedangkan variabel lokal dimulai dari offset `FRAME_HEADER_SIZE`. Pada fungsi, assignment ke nama fungsi dibaca kembali sebagai nilai return dan diikuti instruksi `RET 1 psze`; pada prosedur, instruksi akhirnya adalah `RET 0 psze`. Operand `psze` dipakai interpreter untuk membersihkan argumen setelah subprogram selesai.

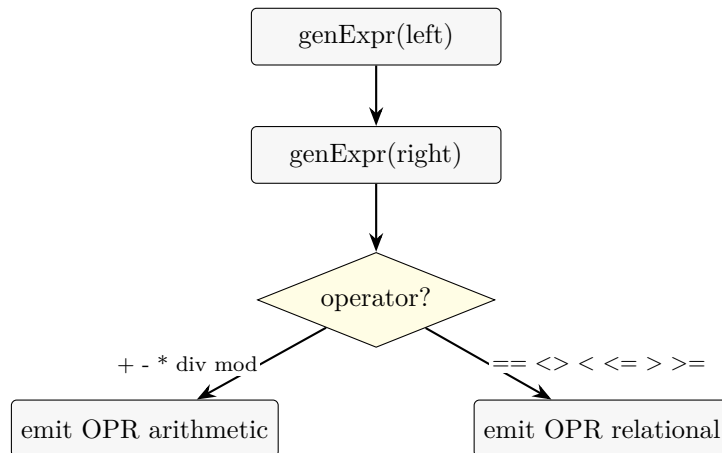


Figure 7: Skema translasi ekspresi biner pada code generator.

2.5 Interpreter

Komponen **Interpreter** diimplementasikan sebagai modul header dan library terpisah. Interpreter menyimpan daftar instruksi, stack eksekusi, serta pointer `ip` dan `bp`. Setiap opcode memiliki handler terpisah untuk operasi literal, load, store, call, jump, conditional jump, operasi OPR, dan return.

Stack frame menggunakan tiga slot awal. Pada awal eksekusi, interpreter mendorong frame

header global berisi tiga nol. Pada pemanggilan subprogram, **CAL** mendorong static link, dynamic link, dan return address, lalu **INT** menambahkan ruang lokal.

Struktur frame ditunjukkan pada Gambar 8. Tiga slot pertama (*offset* 0–2) berisi static link, dynamic link, dan return address. Variabel lokal menempati *offset* positif mulai dari **FRAME_HEADER_SIZE** (3), sedangkan parameter berada pada *offset* negatif relatif terhadap BP karena di-*push* oleh pemanggil sebelum **CAL**.

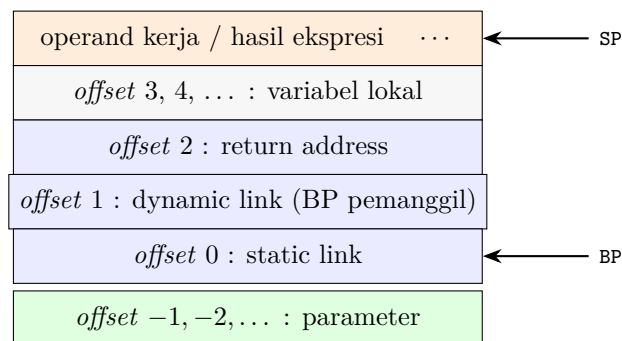


Figure 8: Struktur stack frame Arion: header (SL, DL, RA), variabel lokal pada *offset* positif, dan parameter pada *offset* negatif.

Alamat variabel dihitung lewat `resolveAddress(level, address)`. Nilai `level` menyatakan berapa banyak static link yang perlu ditelusuri dari frame aktif untuk mencapai frame deklarasi variabel. Mekanisme ini penting untuk *nested scope*: Gambar 9 memperlihatkan rantai static link ketika prosedur **Inner** (level 2) mengakses variabel milik **Outer** (level 1) atau variabel global (level 0).

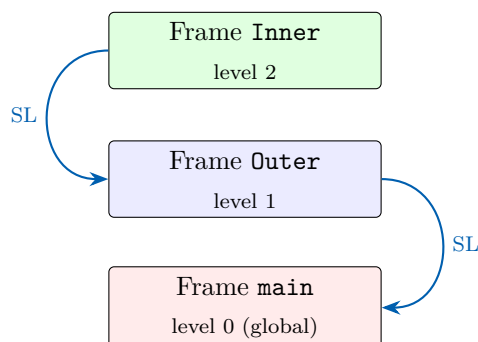


Figure 9: Rantai static link pada *nested scope*. Akses LOD dengan `level` = (level pemakai – level deklarasi) menelusuri rantai ini.

2.6 Error Handling

Pada code generator, error dilaporkan dengan **CodeGenError**. Contoh kasus yang ditangani adalah target assignment yang belum ter-resolve, literal real yang belum didukung oleh stack machine, `readln` yang belum diimplementasikan, dan operator yang belum dipetakan ke **OPR**.

Pada interpreter, error runtime dilaporkan dengan **RuntimeError**. Handler mengecek batas stack, stack kosong sebelum operasi `pop`, target jump di luar range instruksi, frame pointer

invalid, pembagian dengan nol, modulo dengan nol, dan kode OPR yang tidak dikenal. Tabel 4 merangkum skema validasi runtime tersebut.

Kondisi	Mekanisme deteksi	Tindakan
Stack Overflow	cek SP terhadap MAX_STACK_SIZE	RuntimeError: stack overflow
Stack Underflow	cek stack kosong sebelum pop	RuntimeError: stack underflow
Invalid Jump	validasi target JMP/JPC/CAL terhadap rentang instruksi	RuntimeError: invalid jump target
OOB Access	cek alamat pada LOD/STO terhadap batas stack	RuntimeError: out-of-bounds stack access
Division/Modulo by Zero	cek pembagi bernilai nol pada DIV/MOD	RuntimeError: division/modulo by zero
Type Mismatch	atribut tipe pada Decorated AST (statis, di M3)	hentikan kompilasi, pesan error tipe

Table 4: Skema validasi runtime pada interpreter Arion.

Sebagai ilustrasi, Gambar 10 menunjukkan kondisi *stack overflow* akibat rekursi tanpa *base case*: setiap pemanggilan menambah frame baru hingga SP melampaui MAX_STACK_SIZE, dan interpreter melempar *RuntimeError* sebelum memori benar-benar rusak.

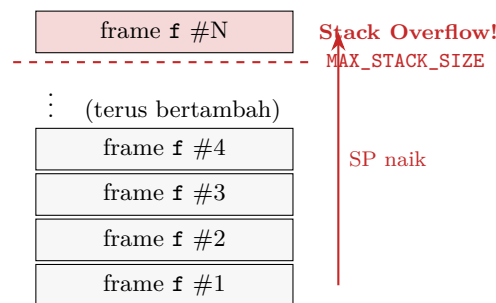


Figure 10: Ilustrasi *stack overflow* pada rekursi tak hingga; interpreter mendeteksi SP melewati batas dan menghentikan eksekusi dengan aman.

2.7 Status Integrasi

Berdasarkan struktur repository saat laporan ini dibuat, komponen Milestone 4 sudah terintegrasi ke pipeline utama. `Makefile` telah memasukkan `src/lib/codegen.cpp` dan `src/lib/interpreter.cpp` ke daftar SRC. Sementara itu, `src/main.cpp` sudah menghubungkan `codegen.hpp` dan `interpreter.hpp`. Setelah lexer, parser, AST builder, dan semantic analyzer selesai, program menghasilkan intermediate code melalui `CodeGenerator`, mencetaknya, lalu menjalankannya melalui `Interpreter` pada bagian `-- Output Program --`.

Dengan demikian, executable utama tidak lagi berhenti pada *Decorated AST*. Output terminal sudah berisi seluruh tahapan: daftar token, parse tree, decorated AST, symbol table, intermediate code, dan output aktual program Arion.

Keterbatasan implementasi yang terlihat dari kode adalah sebagai berikut.

- Eksekusi stack machine masih berbasis `int`; literal `real` belum didukung.

- `writeln` untuk string literal di-skip oleh generator, sehingga output string belum tersedia pada stack machine.
- Assignment baru mendukung variabel sederhana; akses array dan field record belum digenerate.
- `readln` belum didukung pada code generation.
- `case` belum memiliki cabang generator khusus.

3 Testing

Pengujian (*testing*) Milestone 4 disimpan pada direktori `test/milestone-4/`. Setiap testcase memiliki berkas input dan output dengan format `input-{n}.txt` dan `output/output-{n}.txt`. Output pada folder tersebut berbentuk intermediate code teks, sehingga dapat langsung dimasukkan ke laporan dengan `lstinputlisting`.

3.1 Lokasi File Pengujian

- file input: `test/milestone-4/input-1.txt` s.d. `test/milestone-4/input-8.txt`
- file output: `test/milestone-4/output/output-1.txt` s.d. `test/milestone-4/output/output-8.txt`

3.2 Ringkasan Kasus Uji

No	Fokus	Deskripsi singkat	Output
1	Assignment dasar	Assignment integer, operasi penjumlahan, dan <code>writeln</code> .	15
2	<code>if-else</code> dan <code>while</code>	Percabangan dengan <code>JPC/JMP</code> , operasi relasional, dan loop decrement.	0
3	<code>for</code> dan <code>repeat-until</code>	Translasi <code>for-to</code> , akumulasi nilai, dan loop <code>repeat</code> sampai kondisi terpenuhi.	15, 0
4	Function call	Contoh instruksi <code>JMP</code> awal, entry fungsi, <code>CAL</code> , dan <code>RET</code> .	25
5	Nested scope	Contoh static link dan akses variabel pada lexical level berbeda.	—
6	Procedure call dengan parameter	Pemanggilan prosedur <code>add(a,b)</code> , parameter alamat negatif, dan pembersihan argumen lewat <code>RET 0 2</code> .	30, 30
7	Nested procedure dan loop	Prosedur dengan parameter, <code>if-else</code> , <code>while</code> , dan dua kali pemanggilan dari blok utama.	5, 0, 26, 0
8	Aritmatika lengkap	Operasi <code>+</code> , <code>-</code> , <code>*</code> , <code>div</code> , relasional <code><</code> , dan beberapa output.	15, 5, 50, 2, 5

Table 5: Ringkasan kasus uji Milestone 4 beserta output eksekusi yang diharapkan (Kasus 5 tidak memiliki `writeln` sehingga tidak mencetak output).

3.3 Catatan Testcase

Seluruh testcase Milestone 4 sekarang berupa program Arion lengkap yang ditutup dengan `end..`. File output pada repository berisi intermediate code yang dihasilkan generator. Karena `main.cpp` terbaru juga menjalankan interpreter, output final program juga muncul di terminal pada bagian `-- Output Program --`. Pada laporan ini, bukti pengujian ditampilkan dalam bentuk teks agar lebih mudah dibaca dan diverifikasi.

3.4 Hasil Pengujian Intermediate Code

Kasus 1 - Assignment dasar dan `writeln`

Listing 1: `test/milestone-4/input-1.txt`

```
program BasicAssign;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln(b);
end.
```

Listing 2: `test/milestone-4/output/output-1.txt`

```
1 0 INT 0 5
2 1 LIT 0 5
3 2 STO 0 3
4 3 LOD 0 3
5 4 LIT 0 10
6 5 OPR 0 2
7 6 STO 0 4
8 7 LOD 0 4
9 8 OPR 0 14
10 9 RET 0 0
```

Kasus 2 - Percabangan dan `while`

Listing 3: `test/milestone-4/input-2.txt`

```
program TestKeywords;

var
  x, y: integer;

begin
```



```
x := 10;
if x > 5 then
  y := x * 2
else
  y := x div 2;

while y > 0 do
begin
  y := y - 1;
end;

writeln(y);
end.
```

Listing 4: test/milestone-4/output/output-2.txt

```
1 0 INT 0 5
2 1 LIT 0 10
3 2 STO 0 3
4 3 LOD 0 3
5 4 LIT 0 5
6 5 OPR 0 11
7 6 JPC 0 12
8 7 LOD 0 3
9 8 LIT 0 2
10 9 OPR 0 4
11 10 STO 0 4
12 11 JMP 0 16
13 12 LOD 0 3
14 13 LIT 0 2
15 14 OPR 0 5
16 15 STO 0 4
17 16 LOD 0 4
18 17 LIT 0 0
19 18 OPR 0 11
20 19 JPC 0 25
21 20 LOD 0 4
22 21 LIT 0 1
23 22 OPR 0 3
24 23 STO 0 4
25 24 JMP 0 16
26 25 LOD 0 4
27 26 OPR 0 14
28 27 RET 0 0
```

Kasus 3 - for-to dan repeat-until

Listing 5: test/milestone-4/input-3.txt

```
program ForRepeat;

var
  i, sum: integer;
```

```
begin
  sum := 0;
  for i := 1 to 5 do
    begin
      sum := sum + i;
    end;
    writeln(sum);

  repeat
    sum := sum - 1;
  until sum == 0;
  writeln(sum);
end.
```

Listing 6: test/milestone-4/output/output-3.txt

```
1 0 INT 0 5
2 1 LIT 0 0
3 2 STO 0 4
4 3 LIT 0 1
5 4 STO 0 3
6 5 LOD 0 3
7 6 LIT 0 5
8 7 OPR 0 12
9 8 JPC 0 18
10 9 LOD 0 4
11 10 LOD 0 3
12 11 OPR 0 2
13 12 STO 0 4
14 13 LOD 0 3
15 14 LIT 0 1
16 15 OPR 0 2
17 16 STO 0 3
18 17 JMP 0 5
19 18 LOD 0 4
20 19 OPR 0 14
21 20 LOD 0 4
22 21 LIT 0 1
23 22 OPR 0 3
24 23 STO 0 4
25 24 LOD 0 4
26 25 LIT 0 0
27 26 OPR 0 7
28 27 JPC 0 20
29 28 LOD 0 4
30 29 OPR 0 14
31 30 RET 0 0
```

Kasus 4 - Function call

Listing 7: test/milestone-4/input-4.txt

```
program FuncCall;
```

```
var
  r: integer;

function Square(n: integer): integer;
begin
  Square := n * n;
end;

begin
  r := Square(5);
  writeln(r);
end.
```

Listing 8: test/milestone-4/output/output-4.txt

```
1 0 JMP 0 8
2 1 INT 0 4
3 2 LOD 0 -1
4 3 LOD 0 -1
5 4 OPR 0 4
6 5 STO 0 3
7 6 LOD 0 3
8 7 RET 1 1
9 8 INT 0 4
10 9 LIT 0 5
11 10 CAL 0 1
12 11 STO 0 3
13 12 LOD 0 3
14 13 OPR 0 14
15 14 RET 0 0
```

Kasus 5 - Nested scope

Listing 9: test/milestone-4/input-5.txt

```
program NestedScope;

var
  x: integer;

procedure Outer;
var
  x: integer;
  y: integer;

  procedure Inner;
  var
    z: integer;
  begin
    z := x + 1;
```

```
    y := z;
end;

begin
    x := 10;
    y := 20;
    Inner;
end;

begin
    x := 1;
    Outer;
end.
```

Listing 10: test/milestone-4/output/output-5.txt

```
1 0 JMP 0 16
2 1 INT 0 4
3 2 LOD 1 3
4 3 LIT 0 1
5 4 OPR 0 2
6 5 STO 0 3
7 6 LOD 0 3
8 7 STO 1 4
9 8 RET 0 0
10 9 INT 0 5
11 10 LIT 0 10
12 11 STO 0 3
13 12 LIT 0 20
14 13 STO 0 4
15 14 CAL 0 1
16 15 RET 0 0
17 16 INT 0 4
18 17 LIT 0 1
19 18 STO 0 3
20 19 CAL 0 9
21 20 RET 0 0
```

Kasus 6 - Procedure call dengan parameter

Listing 11: test/milestone-4/input-6.txt

```
program FunctionCallTest;
var
    x, y: integer;

procedure add(a, b: integer);
begin
    x := a + b;
    writeln(x);
end;
```

```
begin
  x := 10;
  y := 20;
  add(x, y);
  writeln(x);
end.
```

Listing 12: test/milestone-4/output/output-6.txt

```
1 0 JMP 0 9
2 1 INT 0 3
3 2 LOD 0 -2
4 3 LOD 0 -1
5 4 OPR 0 2
6 5 STO 1 3
7 6 LOD 1 3
8 7 OPR 0 14
9 8 RET 0 2
10 9 INT 0 5
11 10 LIT 0 10
12 11 STO 0 3
13 12 LIT 0 20
14 13 STO 0 4
15 14 LOD 0 3
16 15 LOD 0 4
17 16 CAL 0 1
18 17 LOD 0 3
19 18 OPR 0 14
20 19 RET 0 0
```

Kasus 7 - Nested procedure dan loop

Listing 13: test/milestone-4/input-7.txt

```
program NestedScopeTest;
var
  a, b: integer;

procedure outer(x: integer);
var
  y: integer;
begin
  y := x * 2;
  if x > 5 then
    y := y + 10
  else
    y := y - 1;
  writeln(y);

  while y > 0 do
    begin
      y := y - 1;
```

```
        end;  
        writeln(y);  
    end;  
  
begin  
    a := 3;  
    b := 8;  
    outer(a);  
    outer(b);  
end.
```

Listing 14: test/milestone-4/output/output-7.txt

```
1 0 JMP 0 33  
2 1 INT 0 4  
3 2 LOD 0 -1  
4 3 LIT 0 2  
5 4 OPR 0 4  
6 5 STO 0 3  
7 6 LOD 0 -1  
8 7 LIT 0 5  
9 8 OPR 0 11  
10 9 JPC 0 15  
11 10 LOD 0 3  
12 11 LIT 0 10  
13 12 OPR 0 2  
14 13 STO 0 3  
15 14 JMP 0 19  
16 15 LOD 0 3  
17 16 LIT 0 1  
18 17 OPR 0 3  
19 18 STO 0 3  
20 19 LOD 0 3  
21 20 OPR 0 14  
22 21 LOD 0 3  
23 22 LIT 0 0  
24 23 OPR 0 11  
25 24 JPC 0 30  
26 25 LOD 0 3  
27 26 LIT 0 1  
28 27 OPR 0 3  
29 28 STO 0 3  
30 29 JMP 0 21  
31 30 LOD 0 3  
32 31 OPR 0 14  
33 32 RET 0 1  
34 33 INT 0 5  
35 34 LIT 0 3  
36 35 STO 0 3  
37 36 LIT 0 8  
38 37 STO 0 4  
39 38 LOD 0 3  
40 39 CAL 0 1  
41 40 LOD 0 4  
42 41 CAL 0 1  
43 42 RET 0 0
```

Kasus 8 - Aritmatika dan relasional

Listing 15: test/milestone-4/input-8.txt

```
program ArithmeticTest;  
var  
  a, b, c: integer;  
  
begin  
  a := 5;  
  b := 10;  
  c := a + b;  
  writeln(c);  
  c := b - a;  
  writeln(c);  
  c := a * b;  
  writeln(c);  
  c := b div a;  
  writeln(c);  
  if a < b then  
    writeln(a);  
end.
```

Listing 16: test/milestone-4/output/output-8.txt

```
1 0 INT 0 6  
2 1 LIT 0 5  
3 2 STO 0 3  
4 3 LIT 0 10  
5 4 STO 0 4  
6 5 LOD 0 3  
7 6 LOD 0 4  
8 7 OPR 0 2  
9 8 STO 0 5  
10 9 LOD 0 5  
11 10 OPR 0 14  
12 11 LOD 0 4  
13 12 LOD 0 3  
14 13 OPR 0 3  
15 14 STO 0 5  
16 15 LOD 0 5  
17 16 OPR 0 14  
18 17 LOD 0 3  
19 18 LOD 0 4  
20 19 OPR 0 4  
21 20 STO 0 5  
22 21 LOD 0 5  
23 22 OPR 0 14  
24 23 LOD 0 4  
25 24 LOD 0 3  
26 25 OPR 0 5  
27 26 STO 0 5  
28 27 LOD 0 5  
29 28 OPR 0 14  
30 29 LOD 0 3
```

```
31 30 LOD 0 4
32 31 OPR 0 9
33 32 JPC 0 35
34 33 LOD 0 3
35 34 OPR 0 14
36 35 RET 0 0
```

3.5 Hasil Eksekusi Program

Selain intermediate code, berkas yang sama dijalankan oleh interpreter sehingga menghasilkan output nyata pada bagian -- **Output Program** -- di terminal. Output ini diperoleh dengan menjalankan `make` lalu memasukkan path berkas uji (misalnya `milestone-4/input-4.txt`). Gambar 11 merangkum hasil eksekusi seluruh kasus uji.

```
Kasus 1 (BasicAssign)    -> 15
Kasus 2 (TestKeywords)   -> 0
Kasus 3 (ForRepeat)      -> 15
                        0
Kasus 4 (FuncCall)       -> 25
Kasus 5 (NestedScope)    -> (tanpa output; tidak ada writeln)
Kasus 6 (add prosedur)    -> 30
                        30
Kasus 7 (outer prosedur) -> 5
                        0
                        26
                        0
Kasus 8 (Arithmetic)     -> 15
                        5
                        50
                        2
                        5
```

Figure 11: Output eksekusi aktual kedelapan kasus uji melalui interpreter.

Sebagai contoh penelusuran, Kasus 4 (`r := Square(5)`) menghasilkan 25: pemanggil mendorong argumen 5, `CAL` membentuk frame fungsi, parameter `n` dibaca pada *offset* negatif, hasil `n * n` disimpan ke slot *shadow*, dan `RET 1 1` mengembalikan nilai sekaligus membersihkan satu argumen sebelum `STO` menyimpannya ke `r`.

4 Conclusion

Dari pengerjaan Milestone 4 ini, komponen *Intermediate Code Generator* dan *Interpreter* untuk Arion sudah dibangun dan diintegrasikan sebagai tahap akhir dari pipeline compiler. Generator mampu mengubah sebagian besar AST statement imperatif menjadi instruksi stack machine seperti INT, LIT, LOD, STO, JMP, JPC, CAL, OPR, dan RET. Instruksi tersebut sudah cukup untuk mewakili assignment integer, ekspresi aritmatika, operator relasional, **if-else**, **while**, **for**, **repeat-until**, procedure call, function call dengan return value, nested procedure sederhana, serta output numerik melalui **write/writeln**.

Interpreter juga sudah memiliki arsitektur stack machine dengan instruction pointer, base pointer, stack frame, static link, dynamic link, dan return address. Selain menjalankan instruksi dasar, interpreter sudah memuat error handling runtime untuk stack overflow, stack underflow, akses stack di luar batas, target jump invalid, pembagian dengan nol, dan opcode yang tidak dikenal.

Integrasi utama juga sudah berjalan: setelah semantic analyzer selesai, **main.cpp** mencetak intermediate code dan menjalankan interpreter untuk menghasilkan output final program. Delapan testcase Milestone 4 disiapkan untuk menguji assignment dasar, control flow, loop, function call, procedure call, nested scope, nested procedure, dan operasi aritmatika/relasional.

4.1 Saran

Berdasarkan status implementasi Milestone 4, beberapa saran pengembangan adalah sebagai berikut.

1. **Simpan output lengkap pipeline.** Saat ini file output testcase berisi intermediate code. Ke depan, output final interpreter dapat ikut disimpan agar validasi tidak hanya membandingkan instruksi, tetapi juga hasil eksekusinya.
2. **Dukung tipe non-integer.** Jika interpreter tetap memakai stack integer, string dan real perlu direpresentasikan melalui tabel literal atau variant value agar **writeln('teks')** dan real arithmetic dapat berjalan.
3. **Lengkapi akses komposit.** Tambahkan code generation untuk array indexing, field access pada record, dan assignment tipe komposit.
4. **Tambahkan pengujian runtime.** Selain intermediate code, simpan juga output final interpreter agar pengujian membuktikan dua hal sekaligus: instruksi yang dihasilkan benar dan mesin virtual mengeksekusinya dengan hasil yang sesuai.

References

- [1] A. V. Aho, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006. [Online]. [https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)
- [2] Tim Asisten IF2224, “Arion Interpreter — Guidebook Konvensi Intermediate Code (Milestone 4 & 5),” Laboratorium Ilmu Rekayasa dan Komputasi, STEI ITB, 2026. [Online]. <https://docs.google.com/document/d/1pAy0sLZaSylTLXS1yBc7Snu4dHsAE3fJXbY24DkHeic/edit?tab=t.0>
- [3] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson, 2006.
- [4] N. Wirth, *Algorithms + Data Structures = Programs*. Prentice-Hall, 1976.
- [5] R. Spivak, “Let’s Build A Simple Interpreter,” <https://ruslanspivak.com/lsbasi-part1/>
- [6] “Compiler Design - Intermediate Code Generation,” Tutorialspoint. https://www.tutorialspoint.com/compiler_design/compiler_design_intermediate_code.htm

Appendix

GitHub Repository

<https://github.com/jsndwrd/DCL-Tubes-IF2224-2026>

Task Distribution

NIM	Nama	Kontribusi
13524026	Made Branenda Jordhy	25%
13524028	Muhammad Nur Majiid	25%
13524034	Jason Edward Salim	25%
13524068	Athilla Zaidan Zidna Fann	25%